CÓDIGOCLARO · RUTA KOTLIN

Tema 11 · Excepciones

Guía de estudio · DAM/DAW

try como expresión, throw, require y check.

1. Explicación

En Kotlin, las excepciones son unchecked: el lenguaje no obliga a declararlas en la firma. try puede ser una expresión con un valor en cada rama. throw interrumpe el flujo actual. require expresa que quien llama debe cumplir una precondición y lanza IllegalArgumentException; check señala una condición de estado que el programa esperaba. No conviene capturar Throwable ni ocultar el fallo con un valor que parezca correcto.

2. Ejemplo

Código · kotlin

```kotlin
fun dividir(a: Int, b: Int): Int {
    require(b != 0) { "El divisor no puede ser cero" }
    return a / b
}

fun main() = println(dividir(8, 2))
```

Resultado esperado

```
4
```

3. Errores frecuentes

- Capturar una excepción y devolver un resultado indistinguible de una operación correcta.
- Usar require para validar un fallo externo que debe representarse como resultado del dominio.
- Olvidar que el mensaje de una excepción también debe orientar sin exponer datos sensibles.

4. Ejercicio

Parsea un entero y devuelve su doble; si el texto no es entero devuelve null.

Casos que debe superar

Entrada	Resultado esperado
"4"	8
"0"	0
"x"	null

5. Solución explicada

Código · kotlin

```kotlin
fun dobleSiValido(texto: String): Int? =
    try { texto.toInt() * 2 } catch (_: NumberFormatException) { null }
```

Criterio de aceptación: Se acepta toIntOrNull si el resultado y los límites numéricos mantienen el contrato.

Otras soluciones válidas

- Usar toIntOrNull y decidir sobre null.
- Usar try/catch para traducir NumberFormatException a la respuesta del dominio.

6. Fuentes oficiales

JetBrains / Kotlin — Exception and error handling